



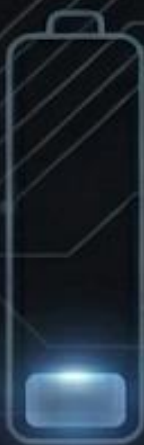
Tattva-AI

Basics to Generative Brilliance

ANN – CNN – VAE/GAN – TRANSFORMERS ...

Module – I

Level – 1 : Building Functional Neural Network



USER



APPLICATION



DEVELOPMENT



EXPERT

BEYOND THE MAGIC: THE MATHEMATICS OF INTELLIGENCE

MODULE – I

Artificial Neural Network

Level – I : User Level

Building Functional Neural Network

What You'll Accomplish in Level 1

By the end of this level, you will understand what an artificial neural network is, how it learns from data, and how to build one from scratch using PyTorch. You'll learn to solve two fundamental types of problems—regression and classification—and follow a systematic pipeline that applies to every neural network you'll ever build.

This isn't about memorizing code. It's about understanding the thinking process behind building intelligent systems that learn from experience.

Artificial Neural Networks: Building from First Principles

Introduction

Artificial Neural Networks (ANNs) are fundamental to modern AI, but they're often introduced in a way that overwhelms beginners with full architecture diagrams before establishing why each component exists.

This document takes a different approach: **start with one neuron, understand it completely, then build complexity incrementally**. By the end, you'll understand not just *what* neural networks are, but *why* they're structured the way they are.

Part 1: The Single Neuron — The Fundamental Building Block

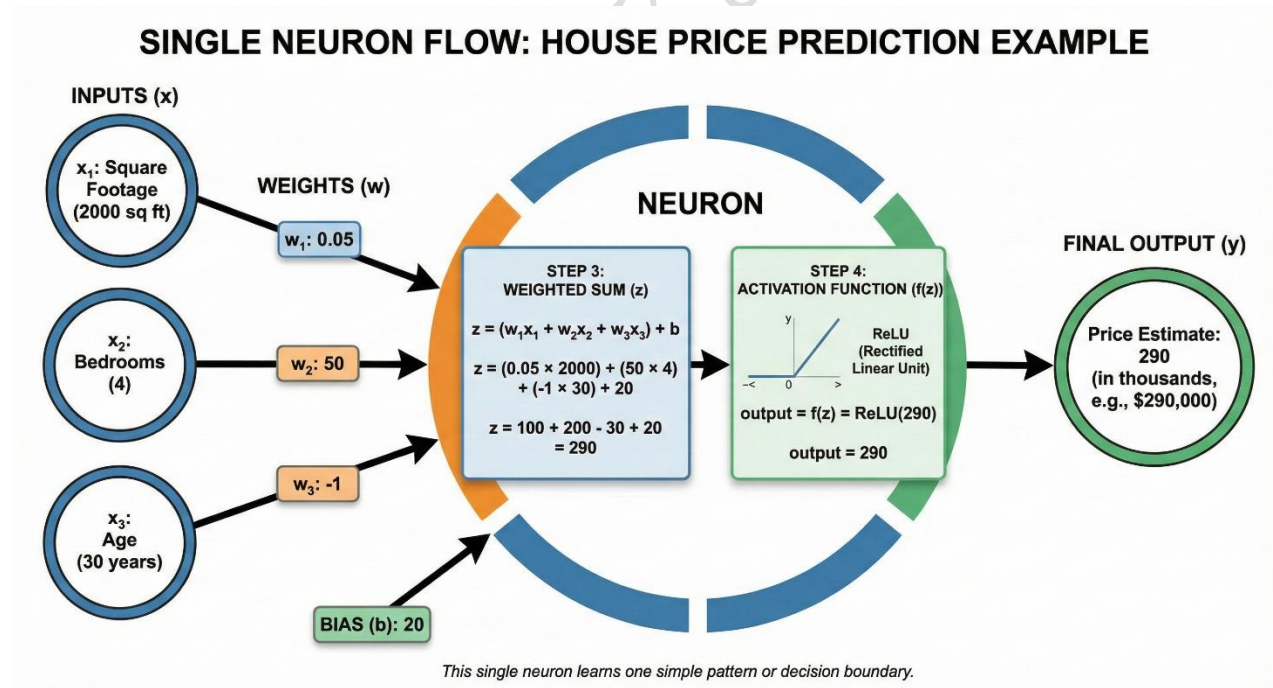
What is a Single Neuron?

A **neuron** is the basic computational unit of a neural network. Think of it as a small machine that: 1. **Receives multiple inputs** 2. **Weighs the importance** of each input 3. **Combines them together** 4. **Applies a decision rule** 5. **Produces a single output**

The Single Neuron Process

INPUT(s) → [NEURON PROCESS] → OUTPUT

Let's say you're building a model to **predict house prices**. Your single neuron might receive: - **Input 1:** Square footage of the house - **Input 2:** Number of bedrooms - **Input 3:** Age of the house



Step 1: Weighing the Inputs

Each input has a **weight** — a number that represents how important that input is to the final decision.

Square footage $\times$ Weight₁ = Weighted Input₁

Bedrooms $\times$ Weight₂ = Weighted Input₂

Age $\times$ Weight₃ = Weighted Input₃

If the weight on square footage is high (say, 2.5), that means square footage is very important to the price prediction. If the weight on age is low (say, 0.1), age has less influence.

These weights are initially random. The neural network will learn the right weights through training.

Step 2: Adding the Bias

In addition to the weighted sum of inputs, each neuron has a **bias** — a single additional number added to adjust the output.

Why? The bias allows the neuron to shift its decision boundary. Without it, the neuron's output would always pass through the origin (0,0), which is often not realistic.

Analogy: If weights are the slope of a line, bias is the y-intercept.

Step 3: The Weighted Sum

The neuron combines all weighted inputs plus the bias into a single number called **z**:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Where: - x_1, x_2, x_3 = inputs (square footage, bedrooms, age) - w_1, w_2, w_3 = weights - b = bias - z = weighted sum

Concrete Example:

Square footage = 2000, Weight = 0.05, Contribution = 100

Bedrooms = 4, Weight = 50, Contribution = 200

Age = 30, Weight = -1, Contribution = -30

Bias = 20, Contribution = 20

Weighted Sum (z) = 290

Step 4: The Activation Function

Here's where it gets interesting. We pass **z** through a **non-linear function** called an **activation function**. This is crucial because it introduces complexity.

$$\text{output} = f(z)$$

Where f is an activation function.

Why Non-linear? This is the key difference between neural networks and simple linear regression.

Without activation functions: No matter how many neurons you add, they're just computing linear combinations of inputs. You could only learn linear relationships (straight lines).

With activation functions: You can learn curved, complex relationships that capture the true patterns in data.

Common Activation Functions:

ReLU (Rectified Linear Unit): - If $z > 0$, output = z - If $z \leq 0$, output = 0 - Simple and effective for hidden layers

Sigmoid: - Outputs a value between 0 and 1 - Smooth curve from 0 to 1 - Useful for binary classification (probability interpretation)

Tanh: - Outputs a value between -1 and 1 - Centered around zero - Good alternative to sigmoid

Example with ReLU:

If $z = 290$, then $\text{ReLU}(290) = 290$

If $z = -5$, then $\text{ReLU}(-5) = 0$

Step 5: The Final Output

After the activation function, the neuron produces its **output**:

FINAL OUTPUT = Activation Function(Weighted Sum + Bias)

FINAL OUTPUT = $f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$

Complete Single Neuron Flow

Inputs: $x_1 = 2000$ (square footage)

$x_2 = 4$ (bedrooms)

$x_3 = 30$ (age)

Weights: $w_1 = 0.05$

$w_2 = 50$

$w_3 = -1$

$b = 20$

Step 1 - Weighted Sum:

$z = (0.05 \times 2000) + (50 \times 4) + (-1 \times 30) + 20$

$z = 100 + 200 - 30 + 20 = 290$

Step 2 - Activation:

output = $\text{ReLU}(290) = 290$

Final Output: Price estimate = 290 (in thousands, e.g., \$290,000)

What Can One Neuron Do?

A single neuron can learn **one simple pattern** or **one decision boundary**. For example: - "Larger houses tend to be more expensive" - "This email looks like spam" - "Is this a cat? (yes/no)"

What Can't One Neuron Do?

A single neuron cannot capture complex relationships like: - "A house's price depends on square footage AND neighborhood AND market conditions" - "This image contains both a cat AND a dog" - "Price depends on the interaction between age and square footage"

This is where we add more neurons.

Part 2: From One Neuron to Many — The Hidden Layer

The Problem: One Neuron is Insufficient

Real-world problems are rarely captured by a single pattern. Consider predicting house prices: - One pattern might be: "Larger houses are more expensive" - Another pattern might be: "Houses in good neighborhoods are more expensive" - Yet another: "Older houses depreciate faster"

A single neuron can only learn one of these patterns. To learn all of them simultaneously, we need **multiple neurons working in parallel**.

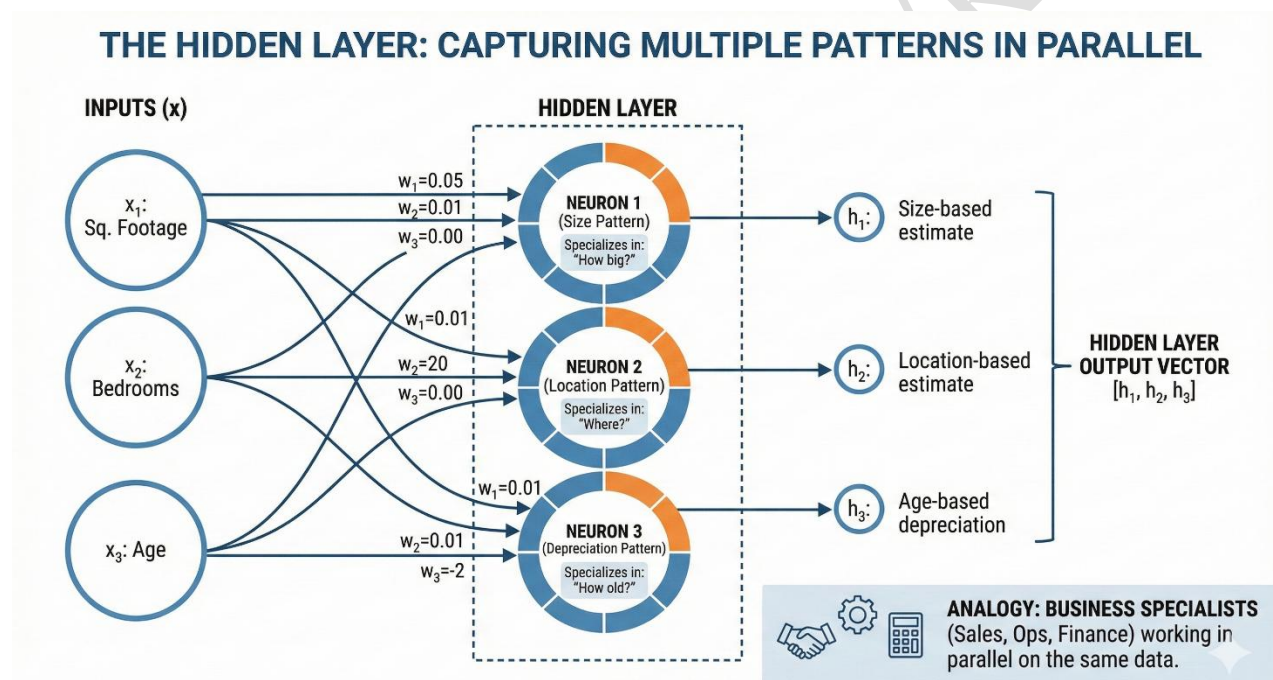
The Solution: Multiple Neurons in Parallel

Instead of one neuron producing one output, we create **multiple neurons receiving the same inputs** but each learning different patterns.

Key Insight: Each neuron has its own set of weights and bias, so even though they receive the same inputs, they produce different outputs.

Concrete Example: Three Hidden Neurons

Let's say we have three neurons learning different aspects of house pricing:



Neuron 1 (Size Pattern): "How big is the house?"

Neuron 2 (Location Pattern): This neuron focuses on: "Where is the house?"

Neuron 3 (Depreciation Pattern):

Why This Works

Each neuron specializes in detecting one aspect of the input space. Together, they create a **distributed representation** of the input that captures multiple patterns simultaneously.

Analogy: In business, you might have: - A Sales specialist (focuses on revenue) - An Operations specialist (focuses on efficiency) - A Finance specialist (focuses on costs)

Each sees the same data but specializes in different aspects. Together, they provide comprehensive understanding.

The Hidden Layer Formally Defined

A **hidden layer** is a collection of neurons (called **units** or **nodes**) that:

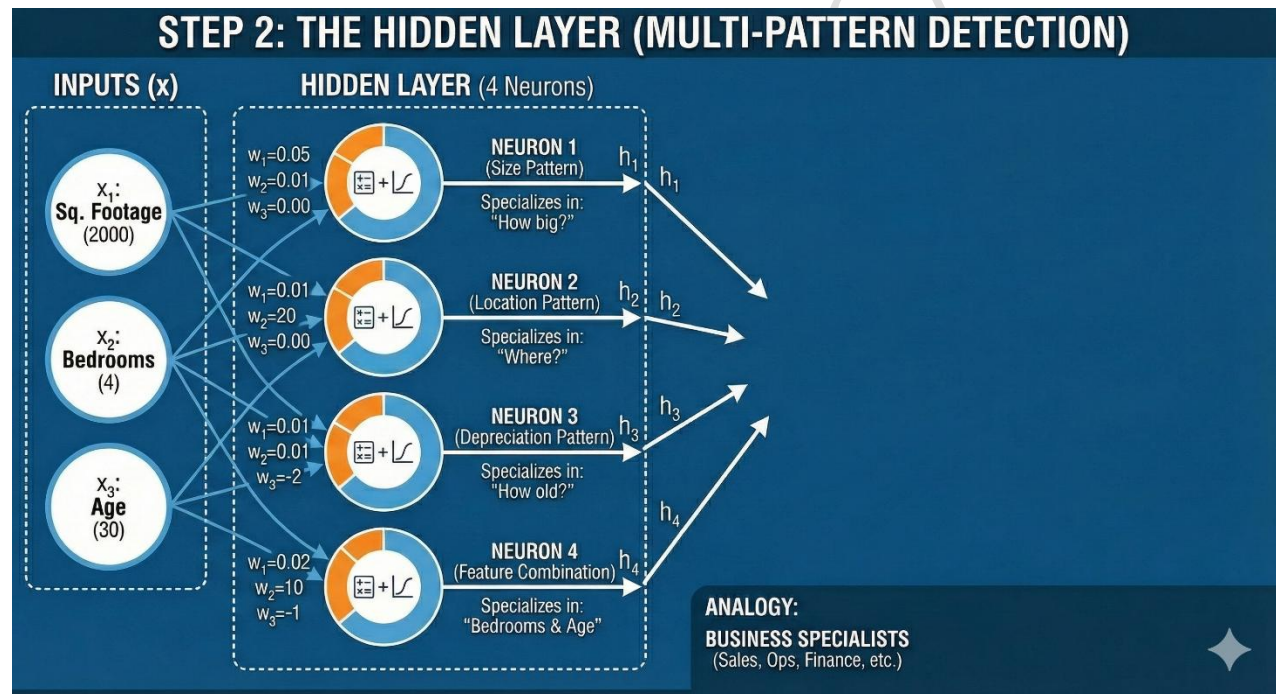
1. Receive the same input vector
2. Each have unique weights and biases
3. Each produce their own output
4. Work in parallel to extract different features from the input

Common sizes:

- Small networks: 10-100 neurons per hidden layer
- Medium networks: 100-1000 neurons
- Large networks: 1000-10,000+ neurons

The larger the hidden layer, the more patterns it can capture—but also the more training data you need and the longer training takes.

Visualizing a Hidden Layer



What Does a Hidden Layer Learn?

Each neuron in the hidden layer learns to recognize a specific feature or pattern:

In Image Processing Example: - Neuron 1 might learn to detect vertical edges - Neuron 2 might learn to detect horizontal edges - Neuron 3 might learn to detect corners - Neuron 4 might learn to detect textures

In Natural Language Example: - Neuron 1 might learn to detect sentiment - Neuron 2 might learn to detect formal vs. casual language - Neuron 3 might learn to detect topic - Neuron 4 might learn to detect urgency

The beauty is: we don't explicitly program these patterns. The network learns them automatically from data.

Part 3: Making Predictions — Why We Need Output Neurons

The Problem: Raw Hidden Outputs Aren't Predictions

The hidden layer produces multiple outputs (h_1, h_2, h_3, h_4 in our example). But our goal is to make a **single, final prediction** (or multiple predictions in specific format).

For example: - **House price**: We need one number: the predicted price - **Email classification**: We need a decision: spam or not spam - **Image recognition**: We need a category: cat, dog, or bird

Raw hidden layer outputs don't directly give us what we need.

The Solution: Output Neurons

We add **output neurons** that receive all hidden layer outputs as inputs and combine them into the final prediction.

The output neuron(s) learn to **combine the feature extractions** from hidden neurons into a final decision.

Analogy: Hidden neurons are like witnesses who each report what they observed. Output neurons are like a judge who weighs all the testimony and makes a final verdict.

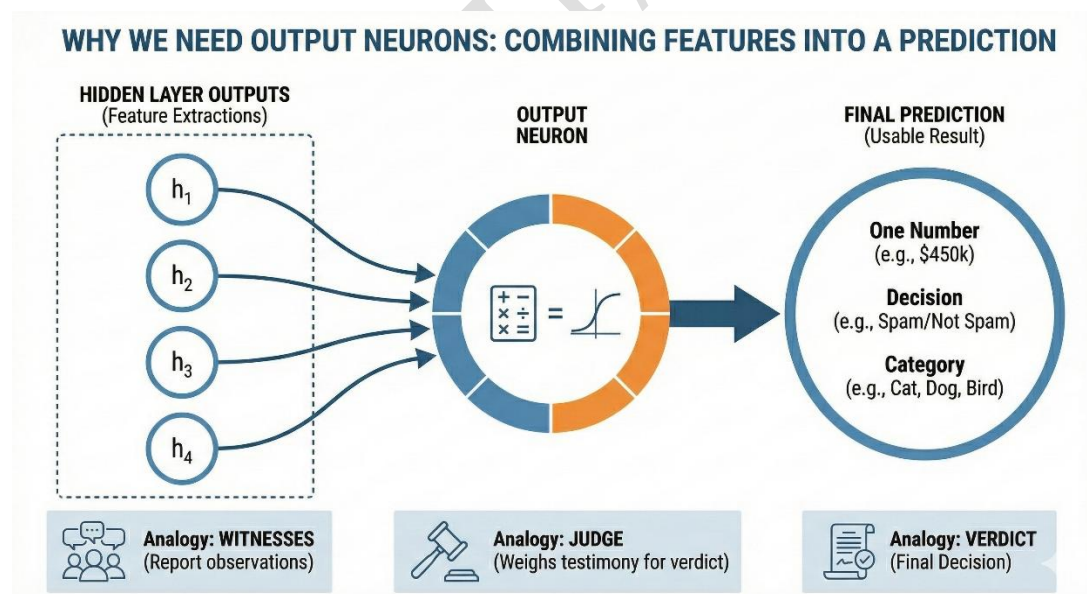
The Architecture Depends on Your Task

Different problems require different numbers and types of output neurons.

Case 1: Regression (Predicting a Continuous Number)

Problem: Predict a continuous value (house price, temperature, stock price)

Solution: One output neuron with **linear activation** (or no activation)



Why Linear Activation? - We want the output to be any real number (positive or negative, small or large) - Linear activation allows this: output = weighted sum (no squashing)

Example:

Hidden outputs: [100, 50, 200, 150]

Weights: [0.5, 0.2, 0.1, 0.3]

Bias: [10]

Output = $(0.5 \times 100) + (0.2 \times 50) + (0.1 \times 200) + (0.3 \times 150) + 10$
= $50 + 10 + 20 + 45 + 10$
= 135 (predicted price in thousands = \$135,000)

Case 2: Binary Classification (Predicting One of Two Categories)

Problem: Classify into one of two categories (spam/not spam, cat/not cat, yes/no)

Solution: One output neuron with **sigmoid activation**

$[h_1, h_2, h_3, h_4] \rightarrow [\text{Output Neuron}] \rightarrow \text{Probability}$
(sigmoid activation)

Why Sigmoid Activation? - Sigmoid squashes output to range [0, 1] - We interpret this as: probability of Class A - If output > 0.5: predict Class A - If output ≤ 0.5: predict Class B

Sigmoid Function: $\sigma(z) = \frac{1}{1+e^{-z}}$

This produces an S-shaped curve: - When z is very negative $\rightarrow \sigma(z) \approx 0$ - When z = 0 $\rightarrow \sigma(z) = 0.5$ - When z is very positive $\rightarrow \sigma(z) \approx 1$

Example:

Hidden outputs: [100, 50, 200, 150]

Weighted sum: 500

Sigmoid(500) ≈ 1.0 (99.999% probability of Class A)

Hidden outputs: [10, 5, 20, 15]

Weighted sum: 50

Sigmoid(50) ≈ 1.0 (almost certain Class A)

Hidden outputs: [-10, -5, -20, -15]

Weighted sum: -50

Sigmoid(-50) ≈ 0.0 (almost certain Class B)

Hidden outputs: [0, 0, 0, 0]

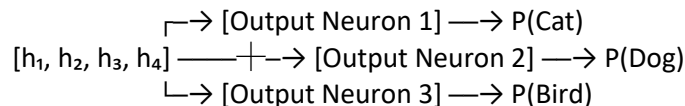
Weighted sum: 0

Sigmoid(0) = 0.5 (50-50 uncertain)

Case 3: Multi-class Classification (Predicting One of 3+ Categories)

Problem: Classify into one of multiple categories (dog/cat/bird, digit 0-9, sentiment positive/negative/neutral)

Solution: Multiple output neurons (one per class) with **softmax activation**



Why Multiple Output Neurons? - Each neuron represents the network's "confidence" in one class - Neuron 1 produces output for "Is it a cat?" - Neuron 2 produces output for "Is it a dog?" - Neuron 3 produces output for "Is it a bird?"

Why Softmax Activation? Softmax does two things: 1. Squashes outputs to [0, 1] 2. **Ensures all outputs sum to 1** (they become probabilities)

Softmax Formula:
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Example: Raw outputs from 3 neurons:

$$z = [2.0, 1.0, 0.1]$$

Softmax applies:

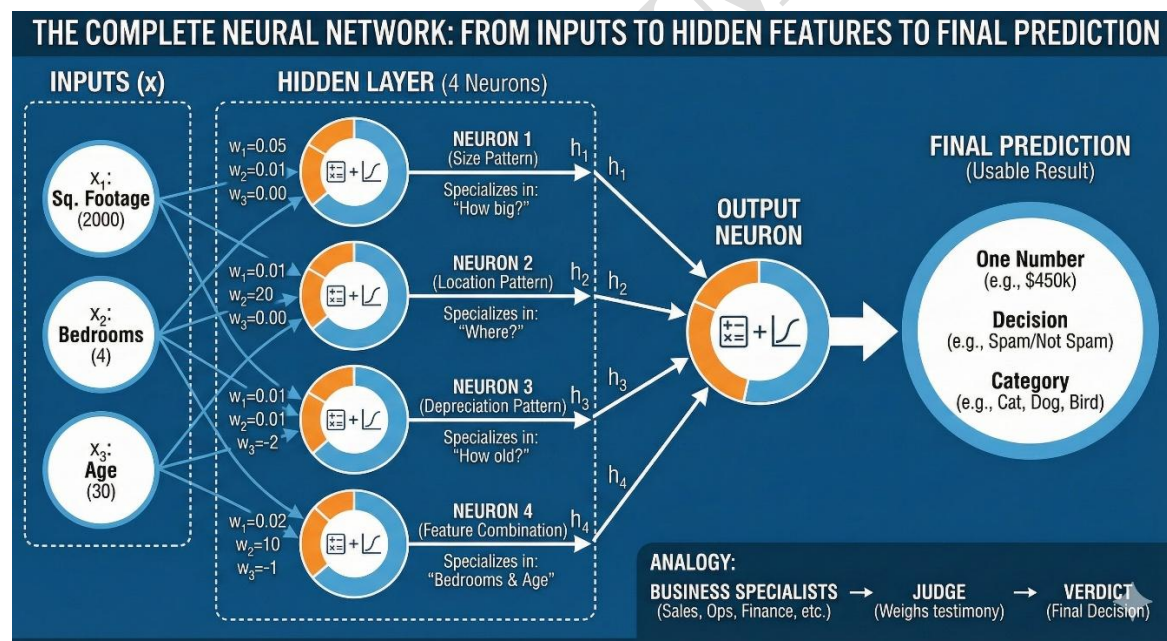
$$P(\text{Cat}) = e^{2.0} / (e^{2.0} + e^{1.0} + e^{0.1}) \approx 0.66 \text{ (66\%)}$$

$$P(\text{Dog}) = e^{1.0} / (e^{2.0} + e^{1.0} + e^{0.1}) \approx 0.24 \text{ (24\%)}$$

$$P(\text{Bird}) = e^{0.1} / (e^{2.0} + e^{1.0} + e^{0.1}) \approx 0.10 \text{ (10\%)}$$

Total = 0.66 + 0.24 + 0.10 = 1.00 ✓ (valid probability distribution)

Prediction: Cat (highest probability)



Summary: Output Layer Design by Task

Task	Output Neurons	Activation	Interpretation
Regression	1	Linear/None	Continuous value
Binary Classification	1	Sigmoid	Probability [0,1]

Task	Output Neurons	Activation	Interpretation
Multi-class Classification	N (one per class)	Softmax	Probability distribution
Multi-label Classification	N (one per label)	Sigmoid (each independent)	Multiple binary decisions

Part 4: Adding Depth — Multiple Hidden Layers

The Problem: One Hidden Layer May Not Be Enough

Some patterns are too complex to learn in a single transformation. They require multiple steps of abstraction.

Example - Image Recognition: - Layer 1 needs to recognize: edges and corners (basic features) - Layer 2 needs to recognize: shapes made from edges (intermediate features) - Layer 3 needs to recognize: objects made from shapes (high-level features) - Output layer needs to: classify objects

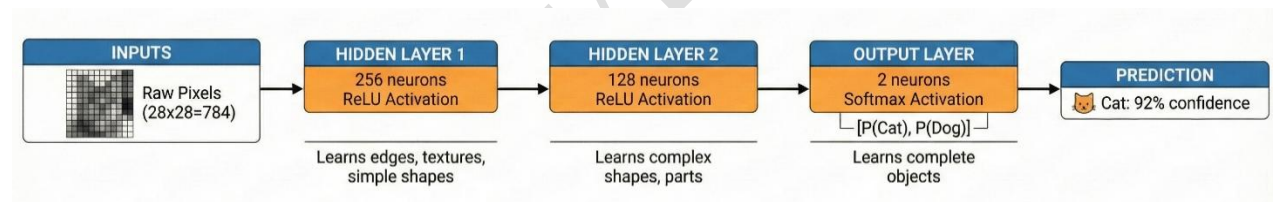
A single hidden layer can't effectively learn all these levels of abstraction simultaneously.

The Solution: Stack Multiple Hidden Layers

INPUT → [Layer 1] → [Layer 2] → [Layer 3] → OUTPUT

Each layer takes the output of the previous layer as input, allowing increasingly complex features to be learned.

The Complete Architecture: From Inputs to Predictions



Why Deeper Networks Learn Better

Deeper networks can learn more abstract features because each layer builds on previous layers' representations.

Example - Face Recognition:

- Layer 1: Detects edges (vertical lines, horizontal lines)
- Layer 2: Detects features (eyes, nose, mouth using edges)
- Layer 3: Detects patterns (face structures using features)
- Layer 4: Classifies (which person, based on structures)

Trying to go directly from raw pixels to "which person" is much harder than going through these intermediate abstraction levels.

Terminology: "Depth" and "Width"

- **Depth:** Number of hidden layers (deeper = more layers)
- **Width:** Number of neurons per layer (wider = more neurons)

Shallow, Wide Network: Deep, Narrow Network:
(1 layer, many neurons) (many layers, few neurons)

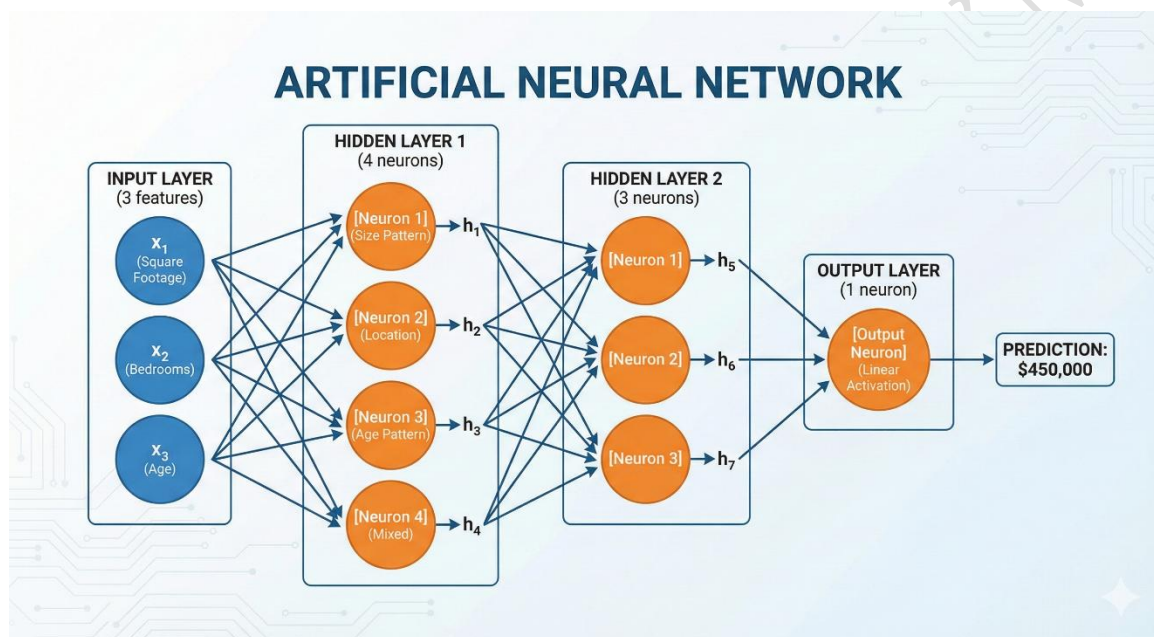
INPUT $\rightarrow$ [1000 neurons] $\rightarrow$ OUTPUT INPUT $\rightarrow$ [100] $\rightarrow$ [100] $\rightarrow$ [100] $\rightarrow$ OUTPUT

General principles: - More depth helps with complex patterns that require multiple transformation steps -
More width helps capture more patterns in a single step - Too deep without enough data can lead to training difficulties - Too wide without enough data leads to overfitting

Part 5: The Complete Neural Network — Bringing It All Together

Full Network Architecture Visualization

Now let's visualize how all components fit together in a complete neural network:



Data Flow Through the Network

Step 1: INPUT

$x = [2000, 4, 30]$ (square footage, bedrooms, age)

Step 2: HIDDEN LAYER 1 COMPUTATION

Each neuron: $h = \text{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + b)$

$$h_1 = \text{ReLU}(0.05 \times 2000 + 0.01 \times 4 + 0.00 \times 30 + 5) = \text{ReLU}(110) = 110$$

$$h_2 = \text{ReLU}(0.01 \times 2000 + 20 \times 4 + 0.00 \times 30 + 10) = \text{ReLU}(100) = 100$$

$$h_3 = \text{ReLU}(0.01 \times 2000 + 0.01 \times 4 + -2 \times 30 + 200) = \text{ReLU}(130) = 130$$

$$h_4 = \text{ReLU}(0.02 \times 2000 + 10 \times 4 + -0.5 \times 30 + 50) = \text{ReLU}(115) = 115$$

Layer 1 Output: [110, 100, 130, 115]

Step 3: HIDDEN LAYER 2 COMPUTATION

$$h_5 = \text{ReLU}(0.3 \times 110 + 0.2 \times 100 + 0.4 \times 130 + 20) = \text{ReLU}(143) = 143$$

$$h_6 = \text{ReLU}(0.2 \times 110 + 0.5 \times 100 + 0.1 \times 130 + 15) = \text{ReLU}(113) = 113$$

$$h_7 = \text{ReLU}(0.1 \times 110 + 0.2 \times 100 + 0.3 \times 130 + 10) = \text{ReLU}(97) = 97$$

Layer 2 Output: [143, 113, 97]

Step 4: OUTPUT LAYER COMPUTATION

$$\text{Price} = \text{Linear}(0.5 \times 143 + 0.3 \times 113 + 0.2 \times 97)$$

$$= \text{Linear}(71.5 + 33.9 + 19.4)$$

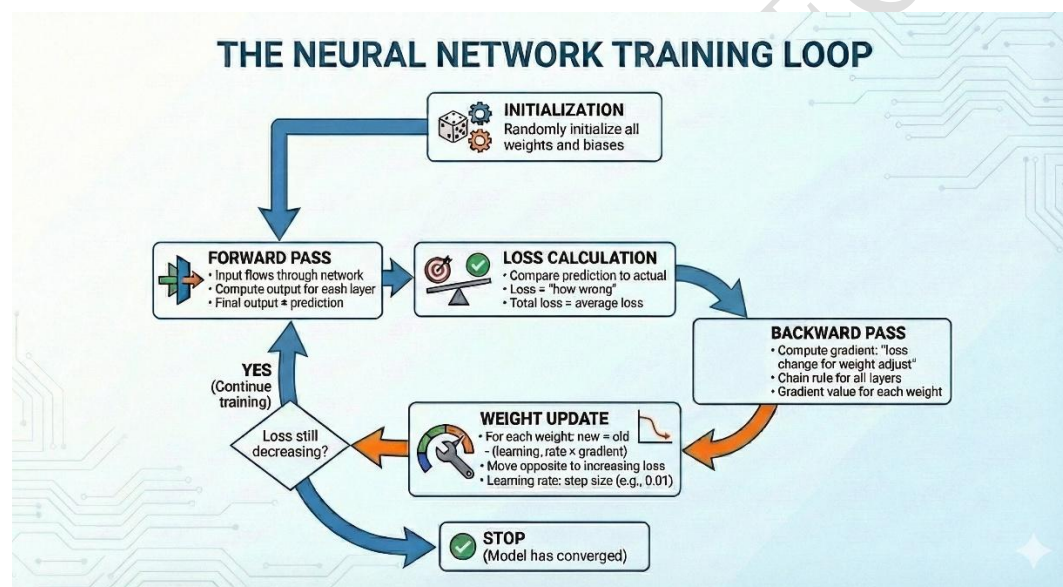
$$= 124.8 \text{ (in hundreds of thousands)}$$

FINAL PREDICTION: \$450,000

The Learning Process: How Networks Adjust Weights

Now that we understand the forward pass (input → prediction), let's understand how networks learn to improve predictions.

The Learning Cycle:



Why This Works: - Each weight adjustment is tiny but in the right direction - After thousands of adjustments, the network converges to good weights - Different problems learn different weights, allowing the same architecture to solve different tasks

Part 6: Connecting to Problem Types

Now that you understand the architecture, let's see how the same structure solves different types of problems by simply changing the output layer.

Problem Type 1: Regression

What we're solving: Predict a continuous number

Examples: - House price prediction - Temperature forecasting - Stock price prediction - Medical outcome prediction (blood pressure, cholesterol level)

Network Design: [Hidden Layers] → [1 Output Neuron, Linear Activation] → Continuous Value

Loss Function: Mean Squared Error (MSE) - Measures: average of (prediction - actual)² - Used because we want to minimize distance from true value

Mean Squared Error (MSE): $MSE = \frac{1}{n} \sum (y_{pred} - y_{actual})^2$ - Penalizes large errors more heavily (quadratic penalty)

Interpretation:

Network output: 450.2

Interpretation: Predicted price is \$450,200

Network output: 98.5

Interpretation: Predicted temperature is 98.5°F

Problem Type 2: Binary Classification

What we're solving: Predict one of two categories

Examples: - Email: Spam or Not Spam - Medical: Disease or Healthy - Financial: Fraud or Legitimate - Image: Cat or Not Cat

Network Design:

[Hidden Layers] → [1 Output Neuron, Sigmoid Activation] → Probability [0,1]

Loss Function: Binary Cross-Entropy - Measures: How well predicted probability matches true label - Used because outputs are probabilities, not raw values

Binary Cross-Entropy: $-[y \log(p) + (1 - y) \log(1 - p)]$ - Where y = actual (0 or 1), p = predicted probability - Penalizes confident wrong predictions heavily

Interpretation:

Network output: 0.92

Interpretation: 92% confident this is Class A, 8% confident this is Class B

Decision: Classify as Class A

Network output: 0.45

Interpretation: 45% confident this is Class A, 55% confident this is Class B

Decision: Classify as Class B (since $0.45 < 0.5$)

Problem Type 3: Multi-class Classification

What we're solving: Predict one of 3+ categories

Examples: - Image recognition: Dog, Cat, Bird, or Hamster - Digit recognition: 0, 1, 2, ..., 9 - Text classification: Politics, Sports, Entertainment, or Science - Medical diagnosis: Disease A, Disease B, Disease C, or Healthy

Network Design:

[Hidden Layers] → [N Output Neurons, Softmax Activation] → Probability Distribution
(One neuron per class)

Loss Function: Categorical Cross-Entropy - Measures: How well probability distribution matches true class
- Used for multiple mutually-exclusive classes

Categorical Cross-Entropy: $-\sum y_i \log(p_i)$ - Where y_i = actual (one-hot encoded), p_i = predicted probability - Penalizes assigning low probability to correct class

Interpretation:

Network outputs (raw): [2.5, 1.8, 0.3]

Network outputs (after softmax): [0.68, 0.25, 0.07]

Interpretation:

Class A: 68% confidence

Class B: 25% confidence

Class C: 7% confidence

Decision: Predict Class A (highest probability)

Part 7: Summary — From Single Neuron to Complete Networks

The Build-Up Process

Stage	Component	Purpose	What Learns
1	Single Neuron	Detect one pattern	One feature/relationship
2	Hidden Layer	Multiple patterns in parallel	Multiple features
3	Multiple Hidden Layers	Hierarchical abstraction	Complex, multi-step patterns
4	Output Layer	Format prediction for task	Conversion from features to answer

Key Insights You Now Understand

1. Why Non-linear Activations Matter - Without them, everything is just linear regression - With them, networks can learn curved, complex relationships

2. Why Multiple Neurons - One neuron = one pattern - Multiple neurons = multiple patterns learned simultaneously

3. Why Multiple Layers - Single layer: transforms raw inputs directly to outputs - Multiple layers: build abstraction hierarchy (simple → complex)

4. Why Output Design Matters - Output neurons and activation function directly depend on what you're predicting - Regression: 1 neuron, linear - Binary classification: 1 neuron, sigmoid - Multi-class: N neurons, softmax

5. Why Weights are Learnable - Initially random - During training, adjusted to minimize error - After training, capture patterns in data - Different problems learn different weights

Next Steps

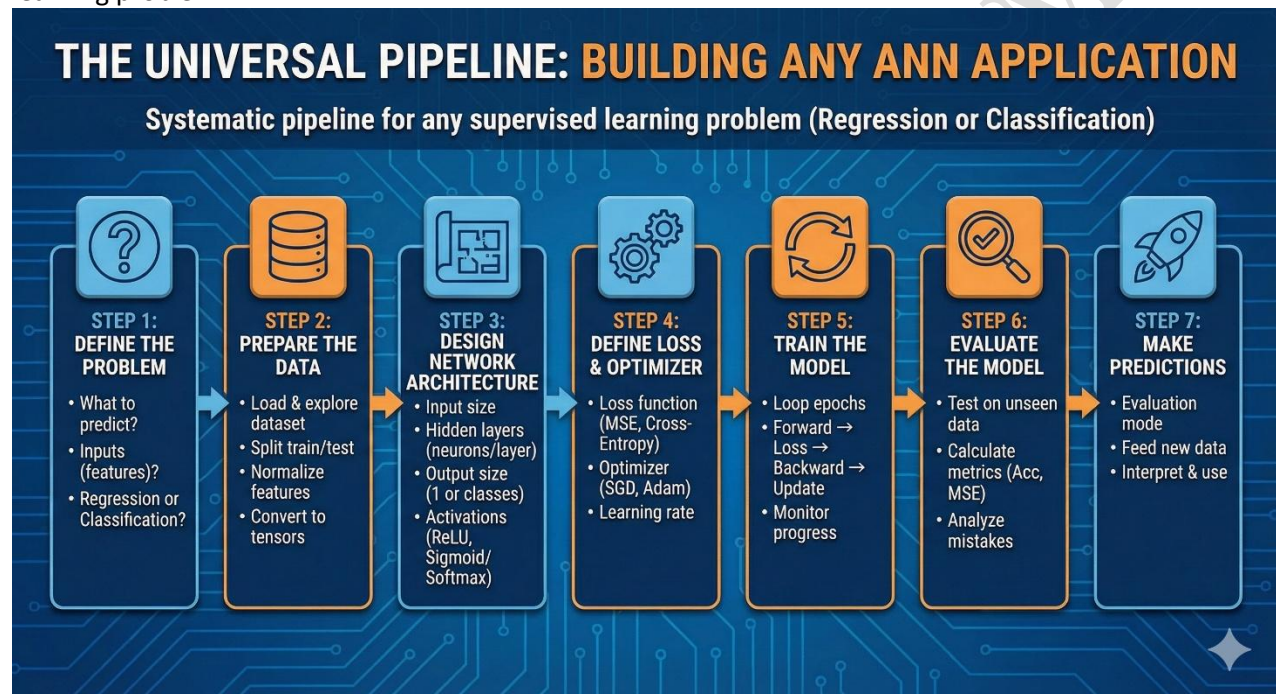
This foundation prepares you to:

1. **Build neural networks in PyTorch** (implement the concepts in code)
2. **Understand training dynamics** (why models overfit, how to prevent it)
3. **Debug and optimize networks** (why isn't my model working? what can I adjust?)
4. **Design custom architectures** (building the right network for your problem)

The same principles apply whether you're building a simple regression model or a cutting-edge generative AI system.

The Universal Pipeline: Building Any ANN Application

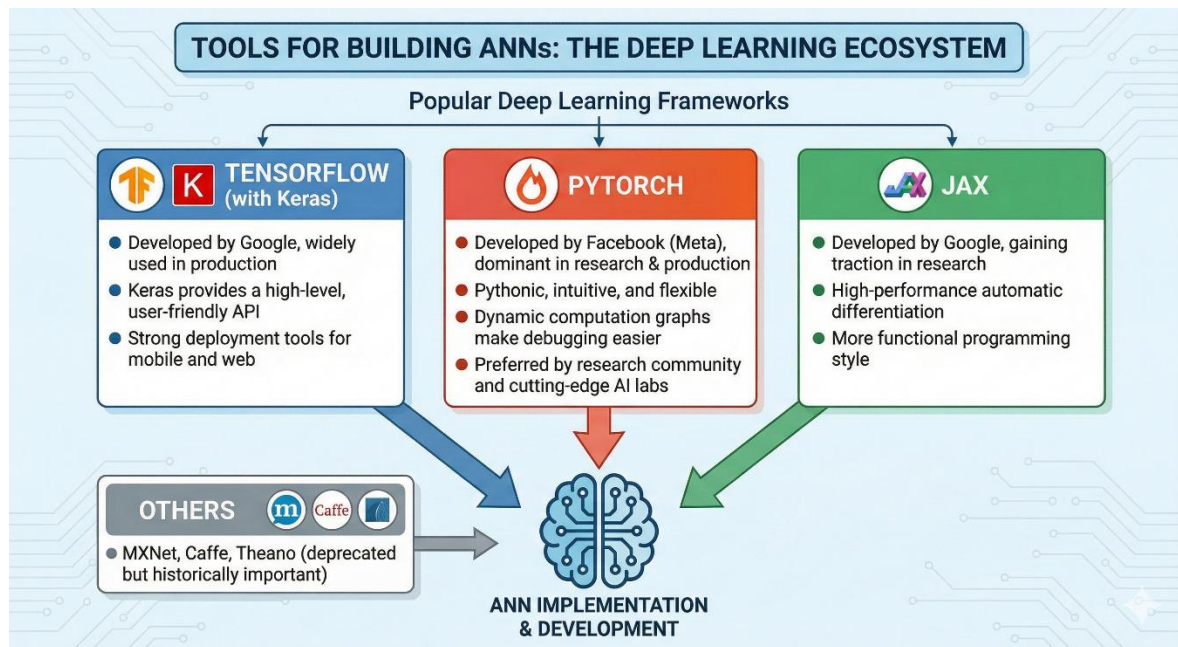
Regardless of whether you're solving regression or classification, building a neural network application follows the same systematic pipeline. Mastering this pipeline means you can tackle any supervised learning problem.



This pipeline is universal. Whether you're building a house price predictor, a spam filter, or eventually contributing to a generative AI system, you'll follow these same seven steps.

Tools for Building ANNs: The Deep Learning Ecosystem

Before we dive into implementation, let's understand the landscape of tools available for building neural networks.



Why We're Using PyTorch

For this course, we're using **PyTorch** for several compelling reasons:^{[8][9][10]}

- Research-Driven:** Most breakthrough papers in generative AI (including the transformers behind ChatGPT and diffusion models behind Stable Diffusion) release PyTorch implementations first
- Intuitive Design:** PyTorch feels like natural Python—if you understand NumPy, you'll understand PyTorch tensors
- Debugging-Friendly:** Dynamic computation graphs mean you can use standard Python debugging tools and inspect operations in real-time
- Industry Momentum:** Companies like OpenAI, Tesla, and Meta use PyTorch for production systems
- Educational Clarity:** PyTorch's explicit design makes it easier to understand what's happening under the hood

PyTorch Prerequisites: What You Need to Know

Before building neural networks in PyTorch, you should be comfortable with:

Python Fundamentals:

- Functions, classes, and object-oriented programming basics
- List comprehensions and basic data structures
- Control flow (loops, conditionals)

NumPy Basics:

- Array creation and manipulation
- Indexing and slicing
- Basic mathematical operations on arrays
- Broadcasting concepts

Basic Mathematics:

- Algebra: Linear equations, variables, functions
- Calculus concepts: Derivatives (what they mean, not necessarily how to compute them manually)
- Basic statistics: Mean, standard deviation, probability

Don't worry if you're rusty on calculus or linear algebra. PyTorch handles the complex mathematics automatically through its autograd system. You need to understand concepts (like "a derivative tells us how much output changes when we change an input"), not perform manual calculations.

Let's begin with your first practical implementation: Understanding Pytorch basics for building the basic models.

PyTorch Essentials: Foundation Before Building

Before you construct your first neural network, you need to be comfortable with PyTorch's fundamental building blocks. These essentials are the vocabulary you'll use to express neural networks in code. This section focuses purely on PyTorch fundamentals—tensors, operations, and the autograd system—everything you need to know before stepping into model building.

1. Tensors: PyTorch's Core Data Structure

A **tensor** is PyTorch's equivalent of NumPy's array—a multi-dimensional container for numerical data. Everything in PyTorch is built on tensors: inputs, weights, biases, predictions, even loss values.^{[\[1\]](#)[\[2\]](#)[\[3\]](#)}

Understanding Tensor Dimensions

Think of tensors by their dimensions:

- **0-D Tensor (Scalar):** A single number: 5, 3.14
- **1-D Tensor (Vector):** A list of numbers: $[^1][^2][^3][^4]$
- **2-D Tensor (Matrix):** A table of numbers: $[[^1][^2], [^3][^4]]$
- **3-D+ Tensors:** Higher-dimensional structures (images, batches of data)

Creating Tensors: Hands-On

```
import torch
# Create tensors from Python lists
tensor_1d = torch.tensor([1, 2, 3, 4])
tensor_2d = torch.tensor([[1, 2, 3], [4, 5, 6]])

print(tensor_1d)
print(tensor_1d.shape) # Output: torch.Size([4])
print(tensor_2d.shape) # Output: torch.Size([2, 3])

# Create tensors with zeros or random values
zeros = torch.zeros(3, 4) # 3x4 matrix of zeros
ones = torch.ones(2, 5) # 2x5 matrix of ones
random_tensor = torch.randn(3, 3) # 3x3 matrix of random values

# Create tensors with specific ranges
range_tensor = torch.arange(0, 10, 2) # [0, 2, 4, 6, 8]

print(f"Zeros shape: {zeros.shape}")
print(f"Random tensor:\n{random_tensor}")

Output:
tensor([1, 2, 3, 4])
```



```
torch.Size([4])
torch.Size([2, 3])
Zeros shape: torch.Size([3, 4])
Random tensor:
tensor([[ -0.4988,  1.0358, -0.0968],
        [-0.6175,  0.5253, -0.1666],
        [-0.6337,  0.5300,  0.4688]])
```

Why this matters: All data flowing through PyTorch—whether it's neural network inputs, weights, or outputs—exists as tensors. Understanding how to create and inspect them is the foundation.

Tensor Data Types (dtypes)

Tensors have data types. The most common:

- `torch.float32` (default): 32-bit floating point numbers—used for neural network computations
- `torch.float64`: 64-bit floating point (double precision)—used when extra precision is needed
- `torch.int32`: 32-bit integers
- `torch.int64`: 64-bit integers—used for indexing
- `torch.bool`: Boolean values

```
# Specify data type explicitly
float_tensor = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float32)
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64)
bool_tensor = torch.tensor([True, False, True], dtype=torch.bool)
```

```
print(f"Float tensor dtype: {float_tensor.dtype}")
print(f"Int tensor dtype: {int_tensor.dtype}")
```

```
# Convert between data types
converted = float_tensor.int() # Convert to int
print(f"Converted dtype: {converted.dtype}")
```

```
Output:
Float tensor dtype: torch.float32
Int tensor dtype: torch.int64
Converted dtype: torch.int32
```

Why this matters: Neural networks primarily work with floating-point numbers for precision. Mismatched data types are a common source of errors. Understanding dtypes helps you debug and optimize code.

Tensor Devices: CPU vs. GPU

PyTorch tensors can live on different devices—CPUs or GPUs. GPU computation is much faster for neural networks:

```
# Check if GPU is available
print(torch.cuda.is_available()) # Returns True or False
print(torch.cuda.get_device_name(0)) # Returns GPU name if available
```

```
# Create tensors on different devices
tensor_cpu = torch.randn(1000, 1000) # Automatically on CPU
print(f"CPU tensor device: {tensor_cpu.device}")
```

```
# If GPU is available, move to GPU
if torch.cuda.is_available():
```

```

    tensor_gpu = tensor_cpu.cuda() # Move to GPU
    # or: tensor_gpu = tensor_cpu.to('cuda')
    print(f"GPU tensor device: {tensor_gpu.device}") # Output: cuda:0
else:
    print("GPU not available, tensor remains on CPU")

# Move back to CPU
tensor_back = tensor_gpu.cpu()
print(f"Moved back to: {tensor_back.device}")

```

Output:

```

True
NVIDIA GeForce RTX 4080
CPU tensor device: cpu
GPU tensor device: cuda:0
Moved back to: cpu

```

Why this matters: During training, you'll need to ensure all tensors (data, weights, gradients) are on the same device. Mismatched devices cause errors.

2. Basic Tensor Operations

Neural networks are fundamentally mathematical operations on tensors. Let's see how PyTorch handles them.

Element-wise Operations

```

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([4.0, 5.0, 6.0])

# Element-wise operations
add = x + y          # [5.0, 7.0, 9.0]
multiply = x * y      # [4.0, 10.0, 18.0]
subtract = x - y      # [-3.0, -3.0, -3.0]
divide = x / y        # [0.25, 0.4, 0.5]
power = x ** 2        # [1.0, 4.0, 9.0]

print(f"Addition: {add}")
print(f"Multiplication: {multiply}")
print(f"Power: {power}")

```

Output:

```

Addition: tensor([5., 7., 9.])
Multiplication: tensor([ 4., 10., 18.])
Power: tensor([1., 4., 9.])

```

In-place Operations

PyTorch allows modifying tensors directly without creating new ones. These operations end with an underscore (_):

```

x = torch.tensor([1.0, 2.0, 3.0])
print(f"Original x: {x}")

# In-place operations modify x directly
x.add_(10) # Add 10 to each element (in-place)
print(f"After add_(10): {x}")

```

```
x.mul_(2)    # Multiply by 2 (in-place)
print(f"After mul_(2): {x}")

# Without underscore, creates a new tensor
y = x + 5    # Creates new tensor, x remains unchanged
print(f"x after x + 5: {x}") # x unchanged
print(f"y = x + 5: {y}")    # new tensor
```

Output:

```
Original x: tensor([1., 2., 3.])
After add_(10): tensor([11., 12., 13.])
After mul_(2): tensor([22., 24., 26.])
x after x + 5: tensor([22., 24., 26.])
y = x + 5: tensor([27., 29., 31.])
```

Why this matters: In-place operations save memory, which matters when working with large neural networks. However, they can sometimes complicate gradient computation, so use them carefully.

Matrix Operations

Matrix multiplication is the heart of neural networks:[2][3]

```
# Matrix multiplication (dot product)
A = torch.tensor([[1.0, 2.0], [3.0, 4.0]]) # 2x2 matrix
B = torch.tensor([[5.0, 6.0], [7.0, 8.0]]) # 2x2 matrix

# Matrix multiplication using @ operator
C = A @ B
print(f"A @ B =\n{C}")

# Equivalent: using torch.matmul()
C_alt = torch.matmul(A, B)
print(f"torch.matmul(A, B) =\n{C_alt}")

# Batched matrix multiplication (common in neural networks)
inputs = torch.randn(32, 10)    # 32 samples, 10 features
weights = torch.randn(10, 5)    # 10 input features → 5 output neurons
outputs = inputs @ weights       # Result: 32x5 (32 samples, 5 outputs)

print(f"Input shape: {inputs.shape}")
print(f"Weight shape: {weights.shape}")
print(f"Output shape: {outputs.shape}")
```

Output:

```
A @ B =
tensor([[19., 22.],
        [43., 50.]])
torch.matmul(A, B) =
tensor([[19., 22.],
        [43., 50.]])
Input shape: torch.Size([32, 10])
Weight shape: torch.Size([10, 5])
Output shape: torch.Size([32, 5])
```

Why this matters: Every neuron performs a weighted sum—exactly what matrix multiplication does. This is the core operation repeated throughout neural networks.

Broadcasting

PyTorch (like NumPy) broadcasts tensors of different shapes to compatible dimensions:

```
# Broadcasting example 1: Adding a scalar
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]]) # 2x2
scalar = 5.0
result = x + scalar # scalar is "broadcast" to match x's shape
print(f"x + 5 =\n{result}")

# Broadcasting example 2: Adding a vector to a matrix
matrix = torch.randn(3, 4)
vector = torch.randn(4) # 1D tensor with 4 elements
result = matrix + vector # vector broadcasts to each row
print(f"Matrix shape: {matrix.shape}")
print(f"Vector shape: {vector.shape}")
print(f"Result shape: {result.shape}")

# Broadcasting example 3: Column and row vectors
col_vector = torch.randn(3, 1) # 3x1
row_vector = torch.randn(1, 4) # 1x4
result = col_vector + row_vector # Broadcasts to 3x4
print(f"Column vector shape: {col_vector.shape}")
print(f"Row vector shape: {row_vector.shape}")
print(f"Result shape: {result.shape}") # 3x4
```

Output:

```
x + 5 =
tensor([[6., 7.],
        [8., 9.]])
Matrix shape: torch.Size([3, 4])
Vector shape: torch.Size([4])
Result shape: torch.Size([3, 4])
Column vector shape: torch.Size([3, 1])
Row vector shape: torch.Size([1, 4])
Result shape: torch.Size([3, 4])
```

Why this matters: Broadcasting allows operations on tensors of different shapes without explicit reshaping. Understanding broadcasting prevents shape-mismatch errors.

Reshaping and Viewing Tensors

```
# Reshape: Change dimensions without changing data
x = torch.randn(6)
x_reshaped = x.reshape(2, 3) # Reshape to 2x3
print(f"Original shape: {x.shape}")
print(f"Reshaped: {x_reshaped.shape}")

# Flatten: Convert multi-dimensional tensor to 1D
x_2d = torch.randn(4, 5)
x_flat = x_2d.flatten()
print(f"Original shape: {x_2d.shape}")
print(f"After flatten(): {x_flat.shape}") # torch.Size([20])

# Unsqueeze: Add a dimension of size 1
x_1d = torch.tensor([1.0, 2.0, 3.0])
```

```
x_2d = x_1d.unsqueeze(0) # Add batch dimension at position 0
print(f"Original shape: {x_1d.shape}")
print(f"After unsqueeze(0): {x_2d.shape}") # [1, 3]

# Squeeze: Remove dimensions of size 1
x_squeezed = x_2d.squeeze()
print(f"After squeeze(): {x_squeezed.shape}") # [^3]

# View: Alternative to reshape (more memory-efficient in some cases)
x = torch.randn(12)
x_view = x.view(3, 4)
print(f"Using view: {x_view.shape}")
```

Output:

```
Original shape: torch.Size([6])
Reshaped: torch.Size([2, 3])
Original shape: torch.Size([4, 5])
After flatten(): torch.Size([20])
Original shape: torch.Size([3])
After unsqueeze(0): torch.Size([1, 3])
After squeeze(): torch.Size([3])
Using view: torch.Size([3, 4])
```

Why this matters: Neural networks expect data in specific shapes. Data often comes in one shape and needs to be reshaped for different operations. Proficiency with reshaping is essential.

Indexing and Slicing

```
x = torch.randn(3, 4, 5)

# Basic indexing
first_element = x[0] # First element along first dimension
print(f"x[0] shape: {first_element.shape}") # [4, 5]

# Slicing
subset = x[0:2, 1:3, :] # Slice along multiple dimensions
print(f"x[0:2, 1:3, :] shape: {subset.shape}") # [2, 2, 5]

# Boolean indexing
condition = x > 0
positive_elements = x[condition]
print(f"Elements > 0: {positive_elements.shape}")

# Advanced indexing
indices = torch.tensor([0, 2])
selected = x[indices] # Select specific indices
print(f"Selected indices shape: {selected.shape}")
```

Output:

```
x[0] shape: torch.Size([4, 5])
x[0:2, 1:3, :] shape: torch.Size([2, 2, 5])
Elements > 0: torch.Size([29])
Selected indices shape: torch.Size([2, 4, 5])
```


Why this matters: You'll frequently need to extract specific parts of tensors—single samples from batches, specific features, or filtered elements.

Useful Operations

```
x = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])

# Reductions (collapse dimensions)
sum_all = x.sum()           # Sum all elements
sum_cols = x.sum(dim=1)     # Sum along dimension 1 (across columns)
mean = x.mean()            # Mean of all elements
max_val = x.max()          # Maximum value
argmax = x.argmax()        # Index of maximum value

print(f"Sum all: {sum_all}")
print(f"Sum across columns (dim=1): {sum_cols}")
print(f"Mean: {mean}")

# Concatenation and stacking
a = torch.randn(3, 2)
b = torch.randn(3, 2)
concat = torch.cat([a, b], dim=1) # Concatenate along dimension 1
print(f"Concatenated shape: {concat.shape}")

stack = torch.stack([a, b], dim=0) # Stack along new dimension
print(f"Stacked shape: {stack.shape}")

# Transpose
x = torch.randn(3, 4)
x_T = x.T           # Transpose (swap last two dimensions)
x_T_alt = x.t()     # Alternative for 2D tensors
print(f"Original shape: {x.shape}")
print(f"Transposed shape: {x_T.shape}")

Output:
Sum all: 21.0
Sum across columns (dim=1): tensor([ 6., 15.])
Mean: 3.5
Concatenated shape: torch.Size([3, 4])
Stacked shape: torch.Size([2, 3, 2])
Original shape: torch.Size([3, 4])
Transposed shape: torch.Size([4, 3])
```

Why this matters: These operations are used constantly in neural network code—computing losses requires reductions, processing batches requires stacking, and weight matrices need transposition.

3. Automatic Differentiation (Autograd): Computing Gradients

This is the engine that makes neural networks learn. **Automatic differentiation (autograd)** is PyTorch's system for automatically computing gradients—derivatives needed for training.^{[3][4][2]}

Understanding Gradients Conceptually

A gradient measures how much an output changes when you change an input slightly. Mathematically, it's a derivative. For neural networks, gradients tell us how changes in weights affect the loss (prediction error).

Enabling Gradient Tracking

```
# Create a tensor that tracks gradients
x = torch.tensor([2.0], requires_grad=True)
print(f"x requires_grad: {x.requires_grad}")

# Create a tensor without gradient tracking
y = torch.tensor([3.0], requires_grad=False)
print(f"y requires_grad: {y.requires_grad}")

# When you perform operations on tensors that require gradients,
# the result automatically requires gradients
z = x + y
print(f"z requires_grad: {z.requires_grad}") # True (because x requires gradients)
```

Output:

```
x requires_grad: True
y requires_grad: False
z requires_grad: True
```

Why this matters: PyTorch only computes gradients for tensors where `requires_grad=True`. This saves memory and computation when you don't need gradients.

Building a Computational Graph

```
# Create variables for which we want gradients
x = torch.tensor([3.0], requires_grad=True)

# Perform operations—PyTorch records each step
y = x ** 2          # y = x2
z = y * 2 + 1       # z = 2y + 1 = 2x2 + 1

print(f"x: {x.item()}")
print(f"y: {y.item()}")
print(f"z: {z.item()}")

# Inspect the computational graph
print(f"z.grad_fn: {z.grad_fn}") # Shows how z was computed
print(f"y.grad_fn: {y.grad_fn}") # Shows how y was computed
print(f"x.grad_fn: {x.grad_fn}") # None (x is leaf node)
```

Output:

```
x: 3.0
y: 9.0
z: 19.0
z.grad_fn: <AddBackward0 object at 0x7f62042dea70>
y.grad_fn: <PowBackward0 object at 0x7f62042dd6c0>
x.grad_fn: None
```

Why this matters: Understanding that PyTorch builds a computational graph helps you visualize how gradients will flow backward during training.

Computing Gradients with Backpropagation

```
# Setup: create variables and compute a result
x = torch.tensor([3.0], requires_grad=True)
y = x ** 2
z = y * 2 + 1

# Compute gradients backward from z
z.backward()

# Access the gradient
print(f"Gradient of z w.r.t. x: {x.grad}") # Output: 12.0

# Interpretation: This gradient tells us how z changes with respect to x
# If we increase x by a small amount, z will increase by approximately 12 times that amount
Output:
Gradient of z w.r.t. x: tensor([12.])
```

What happened step-by-step:

1. `z.backward()` started at `z` and traced backward through the computational graph
2. PyTorch applied the chain rule at each step to compute the gradient
3. The result is stored in `x.grad`

Why this matters: This is exactly how neural network training computes gradients for weight updates—calculate loss, call `loss.backward()`, and PyTorch automatically computes how each weight affects the loss.

Practical Example: Multiple Variables

```
# Multiple variables with gradients
x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([3.0], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)

# Compute a result involving all three
y = w * x + b

# Call backward
y.backward()

# Now we have gradients for all variables
print(f"Gradient of y w.r.t. x: {x.grad}") # 3.0 (the weight)
print(f"Gradient of y w.r.t. w: {w.grad}") # 2.0 (the input)
print(f"Gradient of y w.r.t. b: {b.grad}") # 1.0 (always 1 for bias)

Output:
Gradient of y w.r.t. x: tensor([3.])
Gradient of y w.r.t. w: tensor([2.])
Gradient of y w.r.t. b: tensor([1.])
```

Why this matters: In neural networks, you have many weights, biases, and intermediate values. Backpropagation computes gradients for all of them in one backward pass.

Accumulating vs. Zeroing Gradients

Gradients accumulate by default. Before computing new gradients, you typically zero old ones:

```
x = torch.tensor([2.0], requires_grad=True)

# First backward pass
y = x ** 2
y.backward(retain_graph=True)
print(f"After first backward: x.grad = {x.grad}") # Should be tensor([4.])

# Second backward pass WITHOUT zeroing (gradients accumulate)
z = x ** 3
z.backward(retain_graph=True)
print(f"After second backward (accumulated): x.grad = {x.grad}") # Should be tensor([4.
+ 12. = 16.])

# Zero gradients and perform second backward pass again
x.grad.zero_()
z.backward()
print(f"After zeroing and second backward: x.grad = {x.grad}") # Should be
tensor([12.])
```

Output:

```
After first backward: x.grad = tensor([4.])
After second backward (accumulated): x.grad = tensor([16.])
After zeroing and second backward: x.grad = tensor([12.])
```

Why this matters: During neural network training, you'll call `optimizer.zero_grad()` before each backward pass to ensure gradients are computed fresh for each batch of data, not accumulated from previous iterations.

4. The `no_grad()` Context: Disabling Gradient Computation

Sometimes you don't want PyTorch to compute or track gradients. This is useful when:

- Making predictions (inference)
- Evaluating model performance
- Computing metrics on validation data

```
x = torch.tensor([3.0], requires_grad=True)

# With gradients enabled (default)
y_with_grad = x ** 2
print(f"y_with_grad.requires_grad: {y_with_grad.requires_grad}") # True

# Disable gradients using no_grad()
with torch.no_grad():
    y_no_grad = x ** 2
    print(f"y_no_grad.requires_grad: {y_no_grad.requires_grad}") # False

    # Operations inside no_grad() don't build computational graph
    # This is faster and uses less memory
```

Output:

```
y_with_grad.requires_grad: True
```

```
y_no_grad.requires_grad: False
```

Why this matters: During inference, disabling gradients saves memory and computation since you don't need to track operations for backpropagation. This can reduce memory usage by 50% or more.

Alternative: Detaching Tensors

```
x = torch.tensor([3.0], requires_grad=True)
y = x ** 2

# Detach: breaks the connection from the computational graph
y_detached = y.detach()
print(f"y.requires_grad: {y.requires_grad}") # True
print(f"y_detached.requires_grad: {y_detached.requires_grad}") # False

# Operations on detached tensors don't track gradients
z = y_detached ** 2
print(f"z.requires_grad: {z.requires_grad}") # False

# But you can still use the values
print(f"y_detached value: {y_detached.item()}") # 9.0
```

Output:

```
y.requires_grad: True
y_detached.requires_grad: False
z.requires_grad: False
y_detached value: 9.0
```

Why this matters: detach() is useful when you want to use a tensor's value but prevent gradient computation through it. For example, when you want to use intermediate values without backpropagating through certain layers.

Practical Pattern: Inference

```
# Typical inference pattern
x = torch.randn(10, requires_grad=False) # Input doesn't need gradients
w = torch.randn(10, requires_grad=True)  # Weights do have gradients

# During inference, wrap predictions in no_grad()
with torch.no_grad():
    prediction = x @ w # This doesn't build a computational graph
    print(f"Prediction computed: {prediction.item()}")
# Now prediction is available but has no gradient history
```

Output: (random)
Prediction computed: -1.7171213626861572

Why this matters: This pattern is used everywhere in PyTorch code when you need to make predictions without tracking how they were computed.

5. Important Patterns and Best Practices

Pattern 1: Check if Gradients Exist


```
x = torch.tensor([3.0], requires_grad=True)

# Before backward is called, grad is None
print(f"x.grad before backward: {x.grad}") # None

# After backward, grad contains the value
y = x ** 2
y.backward()
print(f"x.grad after backward: {x.grad}") # tensor(6.0)

# Always check before accessing
if x.grad is not None:
    print(f"Gradient exists: {x.grad}")
else:
    print("Gradients not computed yet")

Output:
x.grad before backward: None
x.grad after backward: tensor([6.])
Gradient exists: tensor([6.])
```

Pattern 2: Safe Backward with Scalars

```
# Backward only works on scalar (single-value) outputs
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]], requires_grad=True)
# If you have a non-scalar output, reduce it first
loss = (x ** 2).sum() # Sum to get a scalar
loss.backward() # Now backward works
print(f"Gradients computed for all elements")
```

Output:
Gradients computed for all elements

Pattern 3: Gradient-Free Operations

```
# Example data for metrics computation
predictions = torch.tensor([0, 1, 2, 1])
labels = torch.tensor([0, 1, 1, 2])

# Example data for preprocessing
data = torch.randn(10) # 10 random numbers

# For operations that don't need gradients
with torch.no_grad():
    # Metrics computation (accuracy for classification, for example)
    accuracy = (predictions == labels).float().mean()

    # Data preprocessing (normalizing tensor)
    normalized = (data - data.mean()) / data.std()
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Normalized data: {normalized}")
    print("These operations are faster and use less memory")
```

Output:
Accuracy: 0.5000

```
Normalized data: tensor([-1.8239,  1.1797, -0.9622,  0.2055,  1.3977, -0.3681,
                        -0.3313,  0.8932,
                        0.2911, -0.4818])
```

These operations are faster and use less memory

Quick Checklist: PyTorch Essentials You Now Have

- ✓ Create tensors and understand shapes
- ✓ Know tensor data types and devices
- ✓ Perform element-wise and matrix operations
- ✓ Reshape and manipulate tensors
- ✓ Enable and disable gradient tracking
- ✓ Understand computational graphs
- ✓ Compute gradients using backpropagation
- ✓ Zero and accumulate gradients
- ✓ Use `no_grad()` and `detach()` strategically
- ✓ Know when and why to disable gradients

You now have all the PyTorch fundamentals needed. The next section builds on these essentials by walking you through building and training your first neural network models.

Building Your First Regression Model: A Complete Pipeline

Now that you understand PyTorch's fundamentals, you're ready to build your first neural network. This section walks you through the complete pipeline for solving a regression problem—predicting a continuous numerical output from input features.

We'll follow the **Universal ANN Pipeline** introduced earlier, applying each step systematically. By the end, you'll have a template you can adapt for any regression problem.

The Problem: Predicting a Simple Continuous Value

Before diving into code, let's define a clear, concrete problem.

Scenario: You want to predict house prices based on a single feature: square footage. In reality, you'd use multiple features, but starting with one feature makes the concepts clear and easy to visualize.

Data Setup:

- **Input:** Square footage of a house (continuous value, e.g., 1500, 2000, 2500)
- **Output:** Price of the house (continuous value, e.g., \$300,000, \$450,000)
- **Relationship:** We assume: $\text{Price} \approx \text{weight} \times \text{square footage} + \text{bias}$

Your neural network will learn this relationship from examples.

Step 1: Define the Problem

Problem Type: Regression (predicting a continuous numerical value)

Input: 1 feature (square footage)

Output: 1 value (price)

What the network needs to learn: The relationship between input and output from data

This is a **regression problem** because:

- We're predicting a continuous number (not a category)
 - The output can be any value in a range
 - We'll use Mean Squared Error (MSE) as the loss function
-

Step 2: Prepare the Data

For this hands-on example, we'll create synthetic data so you can see the complete process without worrying about real dataset complexities.

```
import torch
import torch.nn as nn
import torch.optim as optim

# Create synthetic training data
# Square footage ranges from 800 to 5000
X_train = torch.linspace(800, 5000, 100).unsqueeze(1) # 100 samples, 1 feature
print(f"X_train shape: {X_train.shape}") # torch.Size([100, 1])

# Price = 150 * square_footage + 50000 + noise
# (The network will learn this relationship)
y_train = 150 * X_train + 50000 + torch.randn(100, 1) * 10000
print(f"y_train shape: {y_train.shape}") # torch.Size([100, 1])

# Inspect the data
print(f"\nFirst 5 samples:")
print(f"Square footage: {X_train[:5].squeeze()}")
print(f"Price: {y_train[:5].squeeze()}")
```

Key observations:

- `X_train.unsqueeze(1)` adds a dimension to make it 2D: (100, 1) = (100 samples, 1 feature)
 - Each sample consists of an input feature and a target output
 - There's noise added to make it realistic
-

Step 3: Design the Network Architecture

Now you decide what the network will look like. For this regression problem:

- **Input size:** 1 (one feature: square footage)
- **Output size:** 1 (one prediction: price)
- **Hidden layer size:** 32 neurons (arbitrarily chosen for this simple problem)

```
# Define the neural network architecture
class PricePredictor(nn.Module):
    def __init__(self):
        super(PricePredictor, self).__init__()
```

```

# Layer 1: Transform 1 input feature to 32 hidden neurons
self.linear1 = nn.Linear(1, 32)

# Layer 2: Transform 32 hidden neurons to 16 neurons
self.linear2 = nn.Linear(32, 16)

# Layer 3: Transform 16 neurons to 1 output (the price prediction)
self.linear3 = nn.Linear(16, 1)

# Activation function: ReLU introduces non-linearity
self.relu = nn.ReLU()

def forward(self, x):
    # Forward pass: how data flows through the network
    x = self.linear1(x)      # Linear transformation: 1 → 32
    x = self.relu(x)         # Apply ReLU activation
    x = self.linear2(x)      # Linear transformation: 32 → 16
    x = self.relu(x)         # Apply ReLU activation
    x = self.linear3(x)      # Linear transformation: 16 → 1 (output)
    return x                 # No activation on output (regression)

# Create an instance of the model
model = PricePredictor()
print(f"Model architecture:\n{model}")

# Check the number of parameters
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")

```

Why this architecture:

- **Input layer:** Must match the number of input features (1)
- **Hidden layer (32 neurons):** Gives the network capacity to learn patterns. For this simple problem, 32 is more than enough; for complex problems, you might use 128, 256, or more
- **ReLU activation:** Introduces non-linearity so the network can learn curved relationships
- **Output layer (1 neuron, no activation):** For regression, we don't apply activation to the final layer because we want unrestricted numerical outputs

Step 4: Define Loss Function and Optimizer

The loss function measures "how wrong" predictions are. The optimizer adjusts weights to reduce loss.

```

# Loss function: Mean Squared Error (MSE) is standard for regression
# It measures the average squared difference between predictions and targets
loss_fn = nn.MSELoss()

# Optimizer: Adam (Adaptive Moment Estimation) is a popular choice
# It automatically adjusts learning rates for different parameters
# learning_rate (lr) is a hyperparameter that controls step size
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```
print("Loss function: MSELoss (for regression)")
print("Optimizer: Adam with learning rate 0.001")
```

Why these choices:

- **MSELoss:** Penalizes large errors more than small ones (quadratic penalty). Standard for regression.
 - **Adam:** Works well for most problems. Maintains per-parameter learning rates that adapt during training.
 - **Learning rate 0.001:** Typical starting point. Larger values = faster but riskier learning. Smaller = slower but more stable.
-

Step 5: Training Loop

This is where the learning happens. For each batch of data:

1. Make predictions (forward pass)
2. Calculate loss
3. Compute gradients (backward pass)
4. Update weights

```
# Training parameters
num_epochs = 100 # How many times to loop through the data
batch_size = 10  # How many samples to process before updating weights

# Store losses to visualize learning progress
losses = []

print("Training the model...\n")

for epoch in range(num_epochs):
    # Process data in batches
    for i in range(0, len(X_train), batch_size):
        # Get a batch of data
        X_batch = X_train[i:i+batch_size]
        y_batch = y_train[i:i+batch_size]

        # Step 1: Forward pass - make predictions
        predictions = model(X_batch)

        # Step 2: Calculate loss
        loss = loss_fn(predictions, y_batch)

        # Step 3: Backward pass - compute gradients
        optimizer.zero_grad()    # Clear old gradients
        loss.backward()          # Compute new gradients

        # Step 4: Update weights
        optimizer.step()         # Adjust weights based on gradients

    # Store average loss for this epoch
    with torch.no_grad():
        all_predictions = model(X_train)
```

```

        epoch_loss = loss_fn(all_predictions, y_train)
        losses.append(epoch_loss.item())

    # Print progress every 20 epochs
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}/{num_epochs}, Loss: {epoch_loss.item():.4f}")

print(f"\nTraining complete!")
print(f"Final loss: {losses[-1]:.4f}")

```

Understanding each step:

1. **predictions = model(X_batch)** - Forward pass through the network layers
2. **loss = loss_fn(predictions, y_batch)** - Measure how far predictions are from actual values
3. **optimizer.zero_grad()** - Clear gradients from previous iteration (crucial!)
4. **loss.backward()** - Compute how each weight affects the loss (backpropagation)
5. **optimizer.step()** - Update all weights in the direction that reduces loss

What's happening mathematically: Each weight is adjusted by: $\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$

Step 6: Evaluate the Model

Test your trained model on the training data to see how well it learned:

```

# Put model in evaluation mode (disables dropout if we had any)
model.eval()

# Make predictions without computing gradients (faster, saves memory)
with torch.no_grad():
    all_predictions = model(X_train)
    final_loss = loss_fn(all_predictions, y_train)

print(f"\nModel Evaluation:")
print(f"Final MSE Loss: {final_loss.item():.4f}")

# Calculate RMSE (Root Mean Squared Error - in original units)
rmse = torch.sqrt(final_loss)
print(f"RMSE: ${rmse.item():.2f}")

# Look at a few predictions vs actual
print(f"\nSample predictions vs actual:")
print(f"{'Square Footage':<20} {'Actual Price':<20} {'Predicted Price':<20} {'Error':<20}")
print("-" * 80)

with torch.no_grad():
    for i in range(0, 10, 2): # Show every other sample
        sqft = X_train[i].item()
        actual = y_train[i].item()
        predicted = model(X_train[i:i+1]).item()
        error = abs(actual - predicted)

```

```
print(f"{sqft:<20.0f} ${actual:<19.0f} ${predicted:<19.0f} ${error:<19.0f}")
```

What this shows: How close predictions are to actual values. A good model should have low MSE/RMSE.

Step 7: Making Predictions

Now use your trained model on new data it has never seen:

```
# Create new test data (unseen by the model)
X_test = torch.tensor([
    [1200], # 1200 sq ft
    [2000], # 2000 sq ft
    [3500], # 3500 sq ft
], dtype=torch.float32)

# Make predictions
model.eval() # Switch to evaluation mode
with torch.no_grad(): # No gradients needed for inference
    y_predictions = model(X_test)

# Display predictions
print(f"\nPredictions on new data:")
print(f"{'Square Footage':<20} {'Predicted Price':<20}")
print("-" * 40)
for i in range(len(X_test)):
    sqft = X_test[i].item()
    price = y_predictions[i].item()
    print(f"{sqft:<20.0f} ${price:<19.0f}")
```

Key point: The model now predicts prices for houses it has never seen. This is the true test of learning.

Complete Working Code: Full Pipeline

Here's the entire process in one continuous block:

```
import torch
import torch.nn as nn
import torch.optim as optim

# ===== STEP 1 & 2: Data =====
X_train = torch.linspace(800, 5000, 100).unsqueeze(1)
y_train = 150 * X_train + 50000 + torch.randn(100, 1) * 10000

# ===== STEP 3: Model =====
class PricePredictor(nn.Module):
    def __init__(self):
        super(PricePredictor, self).__init__()
        self.linear1 = nn.Linear(1, 32)
        self.linear2 = nn.Linear(32, 16)
        self.linear3 = nn.Linear(16, 1)
        self.relu = nn.ReLU()

    def forward(self, x):
```

```
        x = self.linear1(x)
        x = self.relu(x)
        x = self.linear2(x)
        x = self.relu(x)
        x = self.linear3(x)
        return x

model = PricePredictor()

# ===== STEP 4: Loss & Optimizer =====
loss_fn = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# ===== STEP 5: Training =====
num_epochs = 100
batch_size = 10

for epoch in range(num_epochs):
    for i in range(0, len(X_train), batch_size):
        X_batch = X_train[i:i+batch_size]
        y_batch = y_train[i:i+batch_size]

        predictions = model(X_batch)
        loss = loss_fn(predictions, y_batch)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if (epoch + 1) % 20 == 0:
        with torch.no_grad():
            print(f"Epoch {epoch+1}, Loss: {loss_fn(model(X_train), y_train):.4f}")

# ===== STEP 6: Evaluation =====
model.eval()
with torch.no_grad():
    print(f"\nFinal MSE: {loss_fn(model(X_train), y_train):.4f}")

# ===== STEP 7: Prediction =====
X_test = torch.tensor([[1500], [2500], [3500]], dtype=torch.float32)
with torch.no_grad():
    predictions = model(X_test)
    print(f"\nPredictions: {predictions.squeeze()}")
```

Adapting This Template: Simple Modifications

The template above is flexible. Here's how to adapt it for different regression problems:

Different Input Size

If you have **5 input features** instead of 1:

```
# Change first layer input size only
self.linear1 = nn.Linear(5, 32) # Changed from 1 to 5
```

Different Output Size

If predicting **3 values** (e.g., price, estimated taxes, closing costs):

```
# Change last layer output size only
self.linear3 = nn.Linear(16, 3) # Changed from 1 to 3
```

Different Network Depth

If you want **more layers**:

```
def __init__(self):
    super().__init__()
    self.linear1 = nn.Linear(1, 64)
    self.linear2 = nn.Linear(64, 32)
    self.linear3 = nn.Linear(32, 16)
    self.linear4 = nn.Linear(16, 8)
    self.linear5 = nn.Linear(8, 1)
    self.relu = nn.ReLU()

def forward(self, x):
    x = self.relu(self.linear1(x))
    x = self.relu(self.linear2(x))
    x = self.relu(self.linear3(x))
    x = self.relu(self.linear4(x))
    x = self.linear5(x)
    return x
```

Different Learning Rate

```
optimizer = optim.Adam(model.parameters(), lr=0.01) # Faster learning (riskier)
# or
optimizer = optim.Adam(model.parameters(), lr=0.0001) # Slower learning (safer)
```

Key Takeaways

- ✓ You've implemented all 7 steps of the ANN pipeline
- ✓ You understand how data flows through the network
- ✓ You've seen how loss decreases as the network learns
- ✓ You can make predictions on new, unseen data
- ✓ You have a template you can modify for any regression problem

The next section applies this exact same pipeline to classification problems—where instead of predicting continuous values, you predict category labels.

Building Your First Classification Model: From Data to Predictions

Now that you've mastered regression, you're ready for the next fundamental problem type: **classification**. Instead of predicting continuous values like prices, you'll predict discrete categories or classes. Let's build a complete classification model using real data from a CSV file, following the same seven-step pipeline you just learned.

Understanding Classification: The Concept

Classification is the task of assigning inputs to one of several predefined categories or classes. [1][2][3]

Key differences from regression:

Aspect	Regression	Classification
Output	Continuous number (e.g., \$450,000)	Discrete category (e.g., "Cat" or "Dog")
Example	Predict house price	Predict email as spam or not spam
Loss Function	MSE, MAE	Cross-Entropy
Output Layer	No activation or linear	Sigmoid (binary) or Softmax (multi-class)
Interpretation	Predicted value	Probability of each class

Binary Classification vs. Multi-class Classification

Binary Classification (2 classes):

- Spam detection: spam vs. not spam
- Medical diagnosis: disease present vs. absent
- Credit approval: approve vs. reject
- **Output layer:** 1 neuron with sigmoid activation → outputs probability between 0 and 1

Multi-class Classification (3+ classes):

- Handwritten digit recognition: 0, 1, 2, ..., 9 (10 classes)
- Email categorization: work, personal, marketing, social (4 classes)
- Flower classification: iris setosa, iris versicolor, iris virginica (3 classes)
- **Output layer:** One neuron per class with softmax activation → outputs probability distribution across all classes

For this hands-on example, we'll use the **Iris dataset**—a classic classification problem with 3 classes (flower types).

The Problem: Iris Flower Classification

Dataset: Iris flowers with measurements and species labels

Input features:

1. Sepal length
2. Sepal width
3. Petal length
4. Petal width

Output classes:

- Iris Setosa (class 0)
-

- Iris Versicolor (class 1)
- Iris Virginica (class 2)

This is a **multi-class classification** problem: given 4 measurements, predict which species of iris it is.

Step 1 & 2: Load and Prepare Data from CSV

First, let's load the Iris dataset from a CSV file and prepare it for training.

```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# ===== Load Data from CSV =====
# Download iris.csv from: https://archive.ics.uci.edu/ml/datasets/iris
# Or create it locally. For this example, we'll use pandas to load it.

# If you have the CSV file locally
df = pd.read_csv('iris.csv') # Adjust path as needed
print(f"Dataset shape: {df.shape}")
print(f"\nFirst few rows:\n{df.head()}")
print(f"\nColumn names: {df.columns.tolist()}")
print(f"\nDataset info:")
print(df.info())

# ===== Explore the Data =====
print(f"\nClass distribution:")
print(df['species'].value_counts())

# ===== Separate Features and Labels =====
# Assuming CSV has columns: sepal_length, sepal_width, petal_length, petal_width,
# species

X = df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']].values
y = df['species'].values

# Convert class names to integer labels
# Assuming species are: 'Setosa', 'Versicolor', 'Virginica'
species_to_label = {'Setosa': 0, 'Versicolor': 1, 'Virginica': 2}
y = torch.tensor([species_to_label[species] for species in y], dtype=torch.long)

print(f"\nFeatures shape: {X.shape}") # (150, 4) - 150 samples, 4 features
print(f"Labels shape: {y.shape}")    # (150,)
print(f"Unique classes: {torch.unique(y).tolist()}") # [0, 1, 2]
```

If you don't have the CSV file yet, here's code to create sample data:

```
# Alternative: Create synthetic Iris-like data if CSV not available
import numpy as np

# Create sample data
np.random.seed(42)
n_samples = 150

# Feature values (sepal_length, sepal_width, petal_length, petal_width)
```

```
X = np.random.randn(n_samples, 4) * 0.7 + np.array([5.8, 3.0, 3.8, 1.2])
X = np.clip(X, 0, 10) # Ensure realistic values

# Labels: 50 samples of each class (0, 1, 2)
y = np.array([0]*50 + [1]*50 + [2]*50)

print(f"Created synthetic data - Features: {X.shape}, Labels: {y.shape}")
```

Normalize Features

Neural networks work better when input features are on similar scales. Normalization brings all features to a standard range.

```
# Normalize features to have mean 0 and standard deviation 1
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X)

# Convert to PyTorch tensors
X_tensor = torch.tensor(X_normalized, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.long) # Labels must be long type

print(f"Normalized X - mean: {X_tensor.mean(dim=0)}, std: {X_tensor.std(dim=0)}")
```

Split into Training and Test Sets

```
# Split data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X_tensor, y_tensor, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
print(f"Training labels distribution: {torch.bincount(y_train)}")
print(f"Test labels distribution: {torch.bincount(y_test)}")
```

Step 3: Design the Network Architecture

For multi-class classification:

- **Input size:** 4 (four features: sepal length, width, petal length, width)
- **Output size:** 3 (three classes: setosa, versicolor, virginica)
- **Output activation:** Softmax (converts outputs to probability distribution)

```
class IrisClassifier(nn.Module):
    def __init__(self):
        super(IrisClassifier, self).__init__()

        # Layer 1: Transform 4 input features to 32 hidden neurons
        self.linear1 = nn.Linear(4, 32)

        # Layer 2: Transform 32 to 16 neurons
        self.linear2 = nn.Linear(32, 16)

        # Layer 3: Transform 16 to 3 output classes
        self.linear3 = nn.Linear(16, 3)

        # Activation function
        self.relu = nn.ReLU()
```



```

def forward(self, x):
    x = self.linear1(x)
    x = self.relu(x)
    x = self.linear2(x)
    x = self.relu(x)
    x = self.linear3(x)      # Output logits (not probabilities yet)
    return x

model = IrisClassifier()
print(f"Model architecture:\n{model}")

# Count parameters
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")

```

Important: Notice the final layer outputs 3 values (logits) without an activation function. PyTorch's CrossEntropyLoss handles the softmax conversion internally, so we don't apply it here.

Step 4: Define Loss Function and Optimizer

For **multi-class classification**, use CrossEntropyLoss:

```

# Loss function for multi-class classification
# CrossEntropyLoss combines log softmax and negative log likelihood
loss_fn = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.Adam(model.parameters(), lr=0.001)

print("Loss function: CrossEntropyLoss (for multi-class classification)")
print("Optimizer: Adam with learning rate 0.001")

```

Why CrossEntropyLoss:

- Measures how far the predicted probability distribution is from the true distribution
- Lower loss = predictions closer to the true class
- Automatically applies softmax internally, so your model doesn't need to

For Binary Classification (if you had only 2 classes):

```
loss_fn = nn.BCEWithLogitsLoss() # Or use CrossEntropyLoss with 2 output neurons
```

Step 5: Training Loop

```

# Training parameters
num_epochs = 100
batch_size = 16

# Store metrics for visualization
train_losses = []
train_accuracies = []

print("Training the model...\n")

for epoch in range(num_epochs):
    total_loss = 0
    correct_predictions = 0

```

```

total_samples = 0

# Process data in batches
for i in range(0, len(X_train), batch_size):
    X_batch = X_train[i:i+batch_size]
    y_batch = y_train[i:i+batch_size]

    # Step 1: Forward pass
    logits = model(X_batch) # Output logits (not probabilities)

    # Step 2: Calculate loss
    loss = loss_fn(logits, y_batch)
    total_loss += loss.item() * len(X_batch)

    # Step 3: Backward pass
    optimizer.zero_grad()
    loss.backward()

    # Step 4: Update weights
    optimizer.step()

    # Track accuracy on this batch
    predictions = torch.argmax(logits, dim=1) # Get predicted class
    correct_predictions += (predictions == y_batch).sum().item()
    total_samples += len(y_batch)

# Average loss and accuracy for this epoch
avg_loss = total_loss / total_samples
avg_accuracy = correct_predictions / total_samples
train_losses.append(avg_loss)
train_accuracies.append(avg_accuracy)

# Print progress
if (epoch + 1) % 20 == 0:
    print(f"Epoch {epoch+1}/{num_epochs}")
    print(f"  Loss: {avg_loss:.4f}")
    print(f"  Accuracy: {avg_accuracy:.4f} ({correct_predictions}/{total_samples}")

print(f"\nTraining complete!")

```

Key differences from regression:

- We use `torch.argmax()` to convert logits to predicted class (the class with highest score)
- We calculate **accuracy**: percentage of correct predictions
- The loss function (`CrossEntropyLoss`) handles softmax conversion internally

Step 6: Evaluate the Model

Evaluate performance on both training and test sets:

```

model.eval() # Switch to evaluation mode

# ===== Evaluate on Training Set =====
with torch.no_grad():
    train_logits = model(X_train)
    train_predictions = torch.argmax(train_logits, dim=1)
    train_accuracy = (train_predictions == y_train).sum().item() / len(y_train)

```

```

    train_loss = loss_fn(train_logits, y_train).item()

print(f"\n{'='*50}")
print(f"Training Set Performance:")
print(f"{'='*50}")
print(f"Loss: {train_loss:.4f}")
print(f"Accuracy: {train_accuracy:.4f} ({int(train_accuracy *
len(y_train))}/{len(y_train)})")

# ===== Evaluate on Test Set =====
with torch.no_grad():
    test_logits = model(X_test)
    test_predictions = torch.argmax(test_logits, dim=1)
    test_accuracy = (test_predictions == y_test).sum().item() / len(y_test)
    test_loss = loss_fn(test_logits, y_test).item()

print(f"\n{'='*50}")
print(f"Test Set Performance:")
print(f"{'='*50}")
print(f"Loss: {test_loss:.4f}")
print(f"Accuracy: {test_accuracy:.4f} ({int(test_accuracy *
len(y_test))}/{len(y_test)})")

# ===== Per-class Performance =====
print(f"\n{'='*50}")
print(f"Per-Class Performance:")
print(f"{'='*50}")
class_names = ['Setosa', 'Versicolor', 'Virginica']
for class_idx in range(3):
    mask = y_test == class_idx
    class_accuracy = (test_predictions[mask] == y_test[mask]).sum().item() /
mask.sum().item()
    print(f"{class_names[class_idx]:<15} Accuracy: {class_accuracy:.4f}")

# ===== Confusion Matrix =====
print(f"\n{'='*50}")
print(f"Detailed Predictions (Test Set):")
print(f"{'='*50}")
print(f"{'True Class':<15} {'Predicted Class':<15} {'Correct':<10}")
print("-" * 40)
for i in range(min(10, len(X_test))):
    true_class = class_names[y_test[i].item()]
    pred_class = class_names[test_predictions[i].item()]
    is_correct = "v" if test_predictions[i] == y_test[i] else "x"
    print(f"{true_class:<15} {pred_class:<15} {is_correct:<10}")

```

Metrics explained:

- **Loss:** CrossEntropyLoss value—lower is better
- **Accuracy:** Percentage of correct predictions
- **Per-class accuracy:** How well the model classifies each flower species
- **Test accuracy > Training accuracy:** Usually indicates underfitting. If test accuracy is much lower, indicates overfitting (learned noise instead of patterns)

Step 7: Making Predictions

Make predictions on new, unseen data:

```
# Create new test samples
new_samples = torch.tensor([
    [5.1, 3.5, 1.4, 0.2], # Likely Setosa
    [6.2, 2.9, 4.3, 1.3], # Likely Versicolor
    [7.1, 3.0, 5.9, 2.1], # Likely Virginica
], dtype=torch.float32)

# Normalize using the same scaler fitted on training data
new_samples_normalized = torch.tensor(
    scaler.transform(new_samples),
    dtype=torch.float32
)

# Make predictions
model.eval()
with torch.no_grad():
    logits = model(new_samples_normalized)

    # Get predicted classes
    predicted_classes = torch.argmax(logits, dim=1)

    # Get prediction probabilities (convert logits to probabilities)
    probabilities = torch.softmax(logits, dim=1)

# Display predictions
print(f"\n{' '*70}")
print(f"Predictions on New Data:")
print(f"{' '*70}")
print(f"{'Features':<30} {'Predicted Class':<20} {'Confidence':<20}")
print("-" * 70)

class_names = ['Setosa', 'Versicolor', 'Virginica']
for i in range(len(new_samples)):
    features_str = f"{new_samples[i].tolist()}"
    predicted_class = class_names[predicted_classes[i].item()]
    confidence = probabilities[i, predicted_classes[i]].item()
    print(f"{features_str:<30} {predicted_class:<20} {confidence:.4f}"
          (f"{confidence*100:.1f}%"))

# Show full probability distribution
print(f"\n{' '*70}")
print(f"Probability Distribution for Each Prediction:")
print(f"{' '*70}")
for i in range(len(new_samples)):
    print(f"\nSample {i+1}: {new_samples[i].tolist()}")
    for class_idx, class_name in enumerate(class_names):
        prob = probabilities[i, class_idx].item()
        print(f"  {class_name:<15} {prob:.4f} ({prob*100:.1f}%")
```

Key points:

- `torch.argmax()` gives the predicted class (index with highest score)
- `torch.softmax()` converts logits to probabilities (sum to 1)
- Confidence is the probability of the predicted class

- Higher confidence = more certain prediction

Complete Classification Pipeline Code

Here's the entire process in one block:

```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# ===== CREATE SAMPLE DATA =====
np.random.seed(42)
n_samples = 150
X = np.random.randn(n_samples, 4) * 0.7 + np.array([5.8, 3.0, 3.8, 1.2])
y = np.array([0]*50 + [1]*50 + [2]*50)

# ===== NORMALIZE & SPLIT =====
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X)
X_tensor = torch.tensor(X_normalized, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.long)

X_train, X_test, y_train, y_test = train_test_split(
    X_tensor, y_tensor, test_size=0.2, random_state=42
)

# ===== MODEL =====
class IrisClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(4, 32)
        self.linear2 = nn.Linear(32, 16)
        self.linear3 = nn.Linear(16, 3)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.linear1(x))
        x = self.relu(self.linear2(x))
        x = self.linear3(x)
        return x

model = IrisClassifier()

# ===== LOSS & OPTIMIZER =====
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# ===== TRAINING =====
num_epochs = 100
batch_size = 16

for epoch in range(num_epochs):
```

```

for i in range(0, len(X_train), batch_size):
    X_batch = X_train[i:i+batch_size]
    y_batch = y_train[i:i+batch_size]

    logits = model(X_batch)
    loss = loss_fn(logits, y_batch)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

if (epoch + 1) % 25 == 0:
    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# ===== EVALUATION =====
model.eval()
with torch.no_grad():
    train_predictions = torch.argmax(model(X_train), dim=1)
    test_predictions = torch.argmax(model(X_test), dim=1)

    train_accuracy = (train_predictions == y_train).float().mean()
    test_accuracy = (test_predictions == y_test).float().mean()

    print(f"\nTrain Accuracy: {train_accuracy:.4f}")
    print(f"Test Accuracy: {test_accuracy:.4f}")

# ===== PREDICTION =====
new_samples = torch.tensor([[5.1, 3.5, 1.4, 0.2]], dtype=torch.float32)
new_samples_normalized = torch.tensor(scaler.transform(new_samples),
dtype=torch.float32)

with torch.no_grad():
    logits = model(new_samples_normalized)
    prediction = torch.argmax(logits, dim=1)[0].item()
    probabilities = torch.softmax(logits, dim=1)[0]

    print(f"\nNew sample prediction: Class {prediction}")
    print(f"Probabilities: {probabilities}")

```

Key Differences: Classification vs. Regression

Aspect	Regression	Classification
Output Layer	1 neuron, no activation	N neurons (N = # classes), softmax
Loss Function	MSELoss, MAELoss	CrossEntropyLoss, BCELoss
Output Interpretation	Direct prediction value	Probability distribution
Getting Prediction	Use output directly	Use argmax() to get class
Metrics	MSE, MAE, R ²	Accuracy, Precision, Recall, F1
Confidence Measure	N/A	Use softmax probabilities

Quick Checklist: Classification Skills You Now Have

- ✓ Understand binary and multi-class classification
- ✓ Load and preprocess data from CSV files
- ✓ Normalize features for neural networks
- ✓ Split data into training and test sets
- ✓ Build a multi-class classifier
- ✓ Use CrossEntropyLoss for classification
- ✓ Calculate accuracy and per-class performance
- ✓ Make predictions with probability outputs
- ✓ Interpret softmax probabilities

You now have mastered both fundamental problem types: regression (continuous predictions) and classification (category predictions). These two skills form the foundation for everything more advanced in deep learning.

Evaluation Metrics: Measuring Model Performance

After training a neural network, you need to understand how well it's actually performing. This section explains the key metrics for both regression and classification with hands-on examples so you can evaluate any model you build. These metrics will be discussed in detail in "Level-3".

For Regression

1. **Mean Squared Error (MSE)**
2. **Root Mean Squared Error (RMSE)**
3. **Mean Absolute Error (MAE)**
4. **R-Squared (R^2)**
5. **Mean Absolute Percentage Error (MAPE)***
6. **Root Mean Squared Logarithmic Error (RMSLE)***

For Classification

1. **Accuracy**
2. **Precision**
3. **Recall (Sensitivity)**
4. **F1-Score**
5. **Confusion Matrix***
6. **Specificity*** - The "true negative rate."
7. **Area Under the ROC Curve (AUC-ROC)**
8. **Precision-Recall Curve & Average Precision***.
9. **Matthews Correlation Coefficient (MCC)*** -.
10. **Classification Report***.

Conclusion

You have now completed Level 1 and built your first working Artificial Neural Network. More importantly, you have understood **how a neural network actually learns**, not just how to run code.

At this stage, you can build models for regression and classification, run the full training process, and make predictions on new data. You have practiced the complete learning cycle: passing data forward through the network, calculating loss, propagating error backward, and updating weights so the model improves. This cycle is the foundation of all deep learning systems.

Level 1 gives you a correct mental picture of neural networks. A model is no longer a black box. You now see it as a system that reduces error step by step through repeated learning. This understanding is far more important than memorizing formulas or APIs.

In the next level, the focus shifts from making a model work to making it work **correctly**. You will learn how to handle data using datasets and dataloaders, how to split data into training and validation sets, and how to identify overfitting by observing model behavior. These skills help ensure that models generalize well to new data instead of simply memorizing what they have seen.

Now you can go for next level to dive into more insights of ANN (Recommended) or You can start with next module that handles images in Neural Networks (ANN & CNN).